

Hyperparameter Tuning in R: Grid Search for Classification Models

Why Tune Hyperparameters?

Every classifier has **hyperparameters** — configuration choices made *before* training that control model complexity and learning behaviour. Unlike model parameters (weights, thresholds), which are inferred from data during training, hyperparameters cannot be learned automatically — they must be chosen deliberately. Choosing them well is the difference between a model that generalises and one that merely memorises.

Define the search space

Identify the hyperparameters to tune and enumerate candidate values for each. Express this as a named list of ranges — this explicitly bounds what the search will explore.

Select the best

The configuration that maximises the chosen performance metric on the validation set is automatically retained as the winning model, ready for final evaluation.

Build the grid

`cla_tune()` enumerates all combinations (the Cartesian product) and evaluates each using the same train/test protocol, ensuring fair, reproducible comparison across configurations.

Validate the result

Re-evaluate the winning model on the held-out test set — data it has never influenced — to obtain an unbiased estimate of generalisation performance.

What changes vs. a plain model?

- Only the model creation line changes: wrap `cla_??()` inside `cla_tune()` with a hyperparameter grid
- The `fit()`, `predict()`, and `evaluate()` calls remain identical
- The experimental protocol (`set.seed`, train/test split) must be fully defined *before* tuning begins
- Tuning does not replace good feature engineering or data quality — it only optimises configuration

 Rule of thumb: start with a small, meaningful grid. Understand each hyperparameter's effect on bias and variance before expanding the search space.

  **Reference:** Bergstra, J. & Bengio, Y. (2012). *Random Search for Hyper-Parameter Optimization*. Journal of Machine Learning Research, 13, 281–305. Available at: <https://jmlr.org/papers/v13/bergstra12a.html>

Hyperparameter Search: `cla_tune`

```
# Loading the package
library(daltoolbox)

# Data and levels
iris <- datasets::iris
slevels <- levels(iris$Species)

# Train/test split
set.seed(1)
sr <- sample_random()
sr <- train_test(sr, iris)
iris_train <- sr$train
iris_test <- sr$test

# Grid search — SVM
tune <- cla_tune(
  cla_svm("Species", slevels),
  ranges = list(
    epsilon = seq(0, 1, 0.2),
    cost = seq(20, 100, 20),
    kernel = c("linear", "radial",
               "polynomial", "sigmoid")
  )
)
model <- fit(tune, iris_train)
```

```
# Evaluation — train
train_pred <- predict(model, iris_train)
train_eval <- evaluate(model,
  adjust_class_label(iris_train, "Species"),
  train_pred)
# accuracy: 0.983

# Evaluation — test
test_pred <- predict(model, iris_test)
test_eval <- evaluate(model,
  adjust_class_label(iris_test, "Species"),
  test_pred)
# accuracy: 0.933
```

Step by step

- Wrap the base model in `cla_tune()` and define `ranges` as a named list
- The grid makes the search space explicit and reproducible
- A larger grid is not automatically better — it only increases search effort
- Training accuracy: **0.983** | Test accuracy: **0.933** — always evaluate by the test set
- Do not interpret the tuned model solely by training metrics
- Define the sampling protocol before starting the tuning process

❏ **Key concept:** `cla_tune` is an **extension of the same experimental structure** — not a separate process. The split, fit, and evaluate workflow remains identical.

Classifiers & Their Key Hyperparameters

Before tuning, it is essential to understand what each hyperparameter controls. Each classifier exposes different levers — tuning without understanding them leads to arbitrary grids and wasted computation.

KNN (k-Nearest Neighbours)

A non-parametric, instance-based classifier that assigns the class of the k nearest training examples. No explicit training phase — the entire dataset is the model.

Key hyperparameter:

- **k** (number of neighbours)
- Small $k \rightarrow$ complex, noisy boundary (high variance)
- Large $k \rightarrow$ smooth boundary (high bias)

Typical range to search: $k = 1$ to 20

MLP (Multilayer Perceptron)

A feedforward neural network with one or more hidden layers. Learns non-linear decision boundaries through backpropagation.

Key hyperparameters:

- **size**: number of neurons in the hidden layer — controls model capacity
- **decay**: L2 regularisation weight — penalises large weights to reduce overfitting

Typical ranges: size = $1-10$, decay = $0-1$

Random Forest (RF)

An ensemble of decision trees trained on bootstrap samples. Reduces variance by averaging predictions across many diverse trees.

Key hyperparameters:

- **mtry**: number of features randomly sampled at each split — controls tree diversity
- **ntree**: number of trees in the ensemble — more trees reduce variance but increase cost

Typical ranges: mtry = $1-3$ (for iris), ntree = $1-10$ (for illustration)

  **Reference:** Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The Elements of Statistical Learning (2nd ed.)*. Springer. Available at: <https://hastie.su.domains/ElemStatLearn/>

Hyperparameter Grids for Other Models

```
# Grids for other classifiers
```

```
# KNN — vary number of neighbors
```

```
ranges_knn <- list(  
  k = 1:20  
)
```

```
# MLP — vary size and regularization
```

```
ranges_mlp <- list(  
  size = 1:10,  
  decay = seq(0, 1, 0.1)  
)
```

```
# Random Forest — vary mtry and ntree
```

```
ranges_rf <- list(  
  mtry = 1:3,  
  ntree = 1:10  
)
```

```
# Usage with cla_tune:
```

```
# tune <- cla_tune(  
#   cla_knn("Species", slevels),  
#   ranges = ranges_knn  
# )  
# model <- fit(tune, iris_train)
```

Step by step

- Define `ranges` as a named list matching the model's hyperparameters
- **KNN:** `k` from 1 to 20 — controls decision boundary smoothness
- **MLP:** `size` (architecture) and `decay` (regularization)
- **RF:** `mtry` (features per split) and `ntree` (ensemble size)
- Understand the meaning of each hyperparameter before expanding the grid

📌 **Key concept:** Understand the hyperparameters **before** building large grids. A well-defined search space is more efficient than an exhaustive grid without criteria.

Best Practices for All Experiments

Preserve the split

Always use `set.seed()` before `sample_random()`. Never compare models trained on different splits.

Evaluate on the test set

The training metric is not the final quality estimate. Always interpret training and test results separately.

Extract levels first

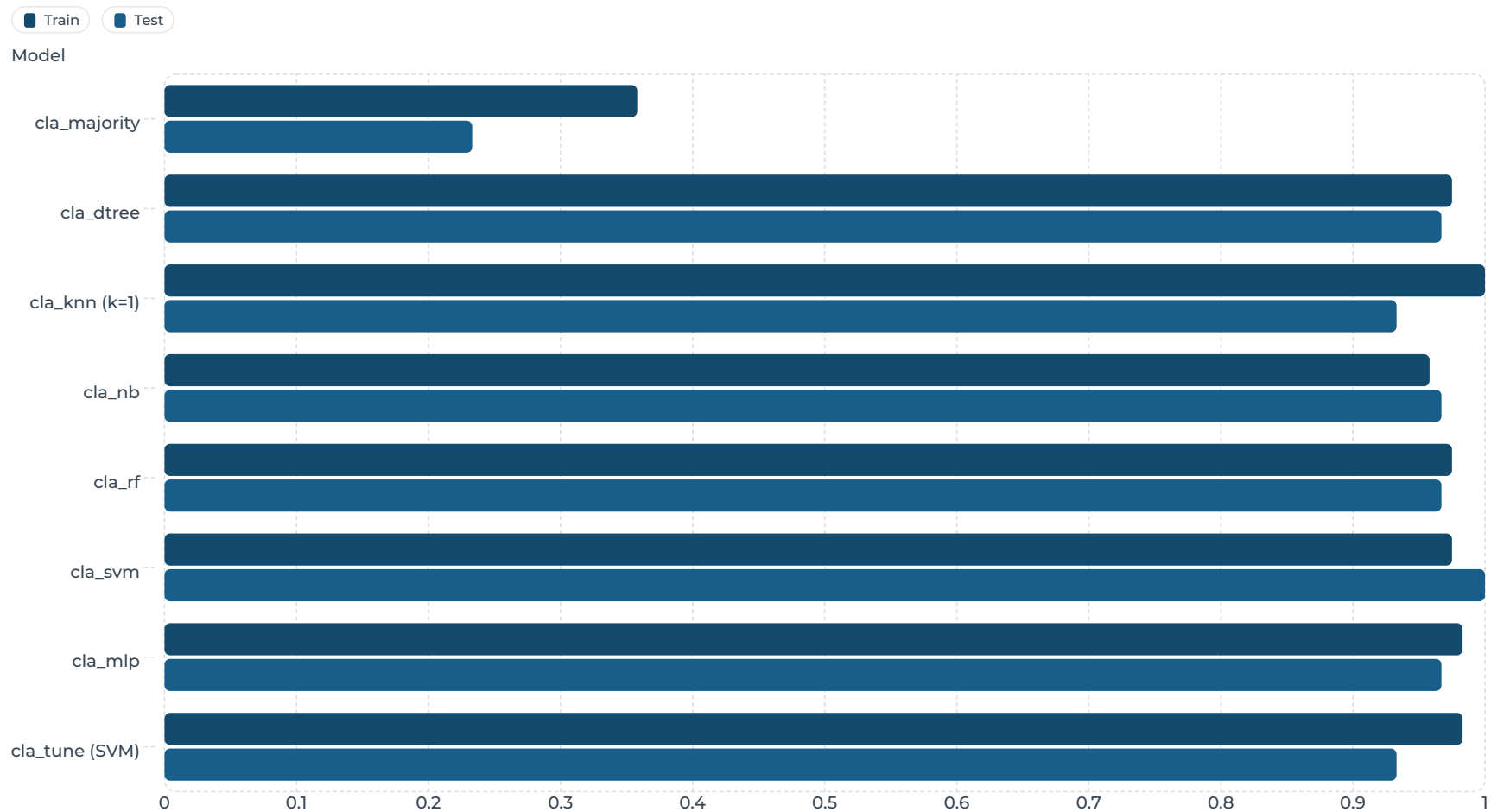
Always run `slevels <- levels(iris$Species)` before creating the model. Forgetting this step is a common mistake.

Adjust the labels

Use `adjust_class_label()` on the response vector before calling `evaluate()`, both on training and test sets.

Performance Comparison

Accuracy on the test set (iris dataset, `set.seed(1)`, same train/test split across all models).



Note that `cla_majority` serves only as a minimum reference baseline. All other models significantly outperform it. The **test** metric is always the definitive quality indicator.

Acknowledgements



Data Analytics Lab

dal.global@grupo.cefet-rj.br | <https://eic.cefet-rj.br/dal>

